

ShapeEnv#

https://pytorch.org/docs/stable/generated/torch.fx.experimental.symbolic_shapes

Description:

Adds a new symbol to the symbolic environment. Given a paired list of placeholders (fake tensors with symbolic sizes) and concrete arguments (regular tensors with real sizes), returns a dictionary mapping each symbol to its real value. So for example, if you have a placeholder with size (s0, s1), binding (2, 4) to it will give you {s0: 2, s1: 4}. This is not guaranteed to bind ALL symbols in the ShapeEnv; we can't bind a symbol if it doesn't occur in any placeholder, and symbols that already have replacements won't get bindings. This is a little duplicative with `evaluate_guards` but it's different enough that it seemed cleanest to make another copy. This assumes the guards are already checked, though if it's cheap we'll check for shenanigans `dict[sympy.Symbol, int]`. Given a sympy expression, computes a `ValueRanges` bound for what values it can be

Function Signature

```
class torch.fx.experimental.symbolic_shapes.ShapeEnv(*,  
should_record_events=None, tracked_fakes=None, **kwargs)  
[source]#  
  
add_var_to_val(expr, val)[source]#  
  
bind_symbols(placeholders, args)[source]#  
  
Return type  
  
bound_sympy(expr, size_oblivious=False)[source]#
```

Returns:

`_GeneratorContextManager[None]`

Detailed Documentation

Documentation

Adds a new symbol to the symbolic environment.

Given a paired list of placeholders (fake tensors with symbolic sizes) and concrete arguments (regular tensors with real sizes), returns a dictionary mapping each symbol to its real value. So for example, if you have a placeholder with size (s0, s1), binding (2, 4) to it will give you {s0: 2, s1: 4}. This is not guaranteed to bind ALL symbols in the ShapeEnv; we can't bind a symbol if it doesn't occur in any placeholder, and symbols

that already have replacements won't get bindings.

This is a little duplicative with `evaluate_guards` but it's different enough that it seemed cleanest to make another copy. This assumes the guards are already checked, though if it's cheap we'll check for shenanigans

`dict[sympy.Symbol, int]`

Given a sympy expression, computes a `ValueRanges` bound for what values it can be

Compare another `ShapeEnv` for equivalence

Break reference cycles.

This destroys the stacks. If you really want to keep them, we just need some way to break references on code objects.

Create a new symbol which is tracked by this `ShapeEnv`

Returns a list of symbolic sizes and strides for the given tensor. We try our best to express stride in terms of the sizes, so as to not introduce new symbolic variables.

`tuple[tuple[IntLikeType, ...], tuple[IntLikeType, ...], IntLikeType]`

Create a `SymBool` object from a sympy boolean expression

Create a `SymFloat` value from a symbolic expression

Create a `SymInt` value from a symbolic expression

If you know what the current hint value of the `SymInt` to be created is, pass it into `hint`. Otherwise, pass `None` and we will make our best guess

Create a symbolic boolean without a hint value

Create a symbolic float without a hint value

Create a symbolic integer without a hint value

Create a symbol with an unspecified value

Compared to standard symbols we do not assume the value is positive, nor do we specialize on zero or one values.

Create a SymInt wrapping a new unspecified symbol

To be used by compile_fx to deserialize symexprs

Union[SymInt, SymFloat, SymBool]

Given an expression, evaluates it, adding guards if necessary. When fallback_value is not None the function returns fallback_value instead of failing with a data dependent error.

Expected to be used with produce_guards_expression(). Evaluates an expression generated by produce_guards_expression for the given concrete args.

Generate guards for a graph's placeholder values and evaluate the guards with args

Given a SymNode, evaluates sym_node.expr, adding guards if necessary.

To be used by compile_fx to evaluate symexprs

Union[int, float, bool]

Format this shape env's guard expressions with optional traceback info if verbose

Freeze this ShapeEnv to stop accumulating guards

A frozen ShapeEnv will ignore any further guards generated on it and only emit a warning which may lead to accuracy problems.

Freeze this ShapeEnv to stop adding deferred runtime asserts.

We will error if you try to install a new runtime assert when it is frozen. This would indicate a lowering violation, or perhaps something we know statically is already True but we are checking it again in a way that is not clearly dischargeable.

Given the symbols in an expression, it returns all the runtime asserts that have those symbols concatenated with all the guards. If symbols is None, it returns all the runtime asserts (and all the guards)

```
tuple['sympy.logic.boolalg.Boolean', ...]
```

Given a expression, it returns a list of predicates that follow from it

```
tuple[tuple['sympy.logic.boolalg.Boolean', sympy.logic.boolalg.BooleanAtom], ...]
```

Returns a list of guard expressions that aren't statically known (i.e. not trivial)

```
list['sympy.logic.boolalg.Boolean']
```

Get a list of guards, but pruned so it only provides guards that reference symints from the passed in input

```
list[torch._guards.ShapeGuard]
```

Adds a guard that orig_expr is True if we can or fall back to adding an assert that is checked at runtime.

orig_expr (sympy.Expr) – Boolean expression to assert is true

msg (str) – Message to display on assertion failure

fx_node (Optional, torch.fx.Node) – node in self.graph corresponding to the expression, if applicable

Indicates that the newly allocated unbacked SymInts are being discarded

Check if a sympy symbol matches the naming convention for unbacked symbols

Temporarily add symbol-level axioms to the ShapeEnv. This is useful when you want to “fork” and have parallel universes of ShapeEnvs. For example, we use this when doing multi-graph compile so we can support various graphs with varying levels of specializations.

This context manager allows for temporarily adding constraints to the shape environment based on a specialization function applied to a symbol associated with a source.

source (Source) – The source of the symbol to specialize

check_fn (Callable[[Symbol], Expr]) – A function that takes a sympy Symbol and returns a sympy expression representing a constraint/specialization to be applied

Like produce_guards_verbose, but only returns the non-verbose python guard expressions (no verbose guards produced.)

Expected to be used with evaluate_guards_expression(). Produces the guards for the given placeholders and returns a string expression to be evaluated by evaluate_guards_expression given concrete values for the placeholders.

Generates a list of guards strings which, when evaluated in a context that defines tensors for all the sources, returns True or False depending on if the guards in the list

evaluated to True or not. Primarily used by Dynamo, but this is also helpful for manual testing of guards (see `evaluate_guards_for_args`)

For convenience in testing, a source is allowed to be a str, in which case we will assume it is a `LocalSource`

`simplified` lets you omit duck sizing, equality and 0/1 guards. This is useful for testing when you don't care about the boilerplate guards, and it may be helpful for user output too (be careful though; some equality guards are nontrivial! It would be nice to get simplified output to print them too). It's private because it's not intended for normal use

Returns guards in python and python with verbose comments (`verbose`) by default.

`list[_ShapeGuardsHelper]`

Apply symbol replacements to any symbols in the given expression.

Used only when `propagate_real_tensors`; registers a value for an unbacked symbol, which can be used last resort to resolve hints.

Use known constraints and replacements to simplify the given expr

Gets a size hint for a given expression from the underlying shapes we had. Does not introduce a guard, so only use this when you can guarantee that your code is still valid for arbitrary shapes (such as optimization decisions)

Context manager to ignore all guards generated inside

`_GeneratorContextManager[None]`